

ANALYSIS AND DESIGN OF A PLATFORM FOR REAL-TIME SPEECH TRANSLATION

JON FRIIS JAKOBSEN 123456789

Masters Project in Software Engineering

June 1887



The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Abstract

Many companies are struggling to combine Java and Machine Learning...

Contents

Contents	ii
1 Introduction	2
1.1 Background and Motivation	2
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Success Criteria	4
1.5 Research Contribution and Scope	5
1.6 Thesis Structure and Question Traceability	5
2 Background	7
2.1 State of the Art in Speech Translation	7
2.2 Microservices and ML Maintainability	7
2.3 Event-Driven Architecture (EDA)	8
2.4 AI Pipeline Components	9
2.5 Challenges in Distributed Architectures	9
2.6 Quality Assurance: From Testing to Evaluation	10
2.7 Synthesis and Design Implications	10
3 Analysis and Design	12
3.1 Architectural Drivers	12
3.2 Quality Attribute Analysis	12
3.3 High-Level System Architecture	12
3.4 Data Modeling and Governance	12
3.5 Service Decomposition	12
3.6 Infrastructure and Deployment Strategy	12
4 Implementation	13
4.1 Orchestrator and Event Flow	13
4.2 Data Management and the Claim Check Pattern	13
4.3 Model Wrapping and "Warm-Loading"	13
4.4 Architectural Guardrails and VAD Integration	13
5 Evaluation	14
5.1 Real-Time Responsiveness (Latency)	14
5.2 Heterogeneous Scalability Testing	14
5.3 Robustness and Chaos Engineering	14
5.4 Architectural Fidelity (Pipeline Quality)	14

<i>CONTENTS</i>	iii
5.5 Modifiability Proof	14
6 Conclusion	15
6.1 Future Work	15
A Requirement Specifications	16
A.1 Functional Requirements	16
A.2 Non-Functional Requirements	17
B Sequence Diagrams	19
C Data Contracts (Avro Schemas)	20
C.1 Base Envelope	20
C.2 Audio Ingestion	20
C.3 Speech Segmentation (VAD)	21
C.4 Translation Output	22
C.5 Synthesis Output (TTS)	23
Bibliography	24

1 Introduction

1.1 Background and Motivation

Real-time speech-to-speech translation is motivated by a straightforward but technically demanding goal: reduce language barriers without forcing users into delayed, text-only, or manually mediated interaction. The societal relevance of multilingual access is well established in educational and public-service contexts, where language inclusion affects participation and access to knowledge [28]. At the same time, market reports and recent industrial overviews indicate strong commercial interest in speech translation, especially for cross-border collaboration, customer support, and meeting assistance [21, 16]. The combination of social demand and commercial momentum makes real-time translation an important applied research problem rather than a narrow laboratory exercise.

Current production-grade offerings are primarily provided by large cloud vendors through tightly integrated service portfolios such as Google Cloud Translation, Microsoft Translator, and the AWS speech stack built around Transcribe, Translate, and Polly [9, 20, 2, 3, 1]. These platforms demonstrate that speech translation has practical value, but they also embody architectural and operational assumptions that are difficult to inspect, adapt, or optimize for smaller actors, research prototypes, and open-source deployment scenarios. This creates a gap between what can be purchased as a managed capability and what can be studied as a transparent, reproducible, and extensible distributed system.

This thesis addresses that gap by investigating an event-driven microservice architecture for real-time speech-to-speech translation. Instead of treating translation as a single opaque model endpoint, the project frames the system as a pipeline of cooperating services for ingress, speech recognition, translation, speech synthesis, orchestration, and evidence collection. That framing is relevant because architectural choices about decoupling, message flow, observability, and artifact handling shape whether the system can achieve low latency, scale under load, and remain robust when components fail [22, 26, 6].

1.2 Problem Statement

The literature on speech translation is rich in model innovation, but comparatively thinner on deployable system architecture. Research has produced strong advances in neural machine translation, speech recognition, and direct

speech-to-speech translation, including transformer-based sequence modeling, large-scale weakly supervised ASR, speech-to-text toolchains, and recent direct translation pipelines [5, 29, 25, 30, 11, 27, 32]. However, these advances do not by themselves explain how multiple AI components should be composed into a distributed platform that must satisfy tail-latency constraints, preserve evidence integrity, and remain maintainable over time.

This gap becomes more pronounced when the system must be evaluated as an architecture rather than a benchmarked model. A real-time speech translation platform must coordinate data flow across ASR, MT, and TTS stages; isolate failures; make queueing and service boundaries observable; and support reproducible measurement of latency and robustness under realistic load [13, 10, 7, 31]. Existing work rarely treats these system-level concerns as the primary object of study. The central problem addressed in this thesis is therefore not only how to translate speech, but how to design and justify an event-driven architecture that makes such translation low-latency, scalable, robust, and empirically assessable.

1.3 Research Questions

The thesis is organized around one main research question and five supporting sub-questions.

Main Research Question

How can an event-driven microservice architecture be designed and implemented to deliver robust and scalable real-time speech-to-speech translation with measurable performance optimization?

Sub-questions

1. What architectural principles and design patterns support an efficient real-time speech translation pipeline?
2. What factors affect the system’s latency and ability to scale?
3. How can these factors be measured and optimized in practice?
4. How can the system be made robust and available even if individual components fail?
5. How can the system’s performance and quality be evaluated and possibly compared to existing solutions?

These questions are intentionally distributed across the thesis rather than answered inside the introduction alone. The current chapter frames the prob-

lem, scope, and evaluation logic. The background chapter establishes the relevant literature and design vocabulary. Later chapters address architectural response, implementation realization, and empirical evaluation in sequence.

1.4 Success Criteria

The thesis evaluates success across architectural, empirical, and methodological dimensions.

Real-Time Success

For this project, *real-time success* is defined by the system’s ability to process speech input and return translated output with an end-to-end latency target of P99 below two seconds under the scoped MVP conditions. This criterion does not claim universal real-time performance across all workloads; instead, it provides a concrete architectural driver that shapes service decomposition, observability requirements, and the later evaluation strategy. In practical terms, low median latency is insufficient if tail latency remains unstable, because the user experience of conversation is dominated by worst-case delays rather than best-case responses.

Scientific Rigor and Evidence Handling

The thesis also treats *scientific rigor* as a success criterion. Performance and robustness claims must be grounded in reproducible evidence generated through the project’s uv-first validation workflow, so that the later implementation and evaluation chapters can refer to repeatable procedures rather than ad hoc one-off measurements. At the same time, manuscript-facing evidence must remain privacy-safe. This means that shareable evidence is restricted to metadata and aggregated observations, such as percentile latency summaries, correlation identifiers, event types, and bounded artifact sizes, while raw transcripts, raw translated text, raw audio, credentials, and full bearer-style URLs are excluded from the manuscript narrative and supporting handoff materials. In particular, latency discussion in this thesis relies on aggregated metrics such as P50, P95, and P99 rather than raw per-request timestamps.

Evaluation Orientation

Success is finally assessed through the combination of latency, scalability, robustness, and quality evaluation. The introduction therefore frames the thesis as a systems investigation in which translation quality matters, but is interpreted together with architectural consequences such as service-boundary overhead, warm-versus-cold execution behavior, and the observability needed to explain measured outcomes. The later evaluation chapter is responsible for

validating these criteria empirically; the introduction only defines the standard against which the system will be judged.

1.5 Research Contribution and Scope

Research Contribution

The contribution of the thesis is a defensible reference architecture for real-time speech-to-speech translation that connects distributed-systems design choices to measurable performance trade-offs. More specifically, the work contributes: (1) an architectural framing for event-driven composition of ASR, MT, and TTS services; (2) a maintainability-focused discussion of service boundaries, observability, and contract evolution in AI pipelines; and (3) an evaluation-oriented treatment of how latency, scalability, and robustness claims can be supported with reproducible and privacy-safe evidence [6, 7, 31]. The contribution is therefore not a new translation model, but a structured architectural and empirical argument for how an open, inspectable platform can be designed and assessed.

Scope Boundaries

The project includes the design and implementation of an event-driven platform built from existing AI components, together with performance, scalability, and robustness evaluation under the scoped MVP conditions. It does not include training new foundation models, exhaustive legal or organizational analysis, full production hardening, or a general-purpose comparison against every commercial alternative. Security is treated here as an evidence-governance and privacy-by-design constraint for manuscript-facing artifacts, while deeper runtime security engineering remains the concern of later implementation and deployment work.

1.6 Thesis Structure and Question Traceability

The remainder of the thesis is organized to answer the research questions progressively. Chapter 2 establishes the literature context for speech translation and event-driven systems. It also introduces AI pipeline maintainability and distributed-systems trade-offs. The Analysis and Design chapter translates that context into architectural drivers, quality attributes, service decomposition, and data-governance choices. The Implementation chapter explains how the platform was realized in practice, including orchestration, artifact handling, and robustness mechanisms. The Evaluation chapter measures the system against the success criteria defined above. The Conclusion then synthesizes the final answer to the main research question.

This structure also makes the sub-question ownership explicit. Sub-question 1 is motivated in the Background chapter and answered architecturally in Analysis and Design. Sub-questions 2 and 3 are framed here first. They are then operationalized through design and implementation and resolved empirically in Evaluation. Sub-question 4 is addressed through implementation and robustness evidence. Sub-question 5 depends on the evaluation and conclusion chapters. The introduction therefore functions as a scope-locking and traceability chapter, ensuring that later arguments remain tied to the original research objective.

2 Background

2.1 State of the Art in Speech Translation

Speech translation research spans both modular pipelines and direct end-to-end approaches. In the modular tradition, speech is transcribed, translated, and optionally synthesized in separate stages, making it possible to combine specialized ASR, MT, and TTS components that can each be improved independently [5, 29, 25, 30]. This decomposition remains attractive in practice because it exposes explicit control points for error handling, observability, fallback behavior, and selective optimization.

More recent work on direct speech-to-speech translation demonstrates that tightly integrated models can reduce intermediate representation overhead and potentially improve naturalness in some settings [11, 27, 32]. Even so, direct approaches do not eliminate the architectural questions relevant to this thesis. A production-oriented platform still needs to orchestrate ingress, buffering, session context, failure handling, artifact persistence, and evaluation evidence. In other words, model architecture and system architecture solve different layers of the same problem.

Industrial offerings confirm the practical importance of the problem space but do not by themselves resolve the research gap. Cloud platforms from Google, Microsoft, and Amazon provide mature translation services, yet they package architectural decisions behind proprietary boundaries [9, 20, 2, 3, 1]. Recent applied studies show that real-time translation is increasingly feasible, but they also underscore the continued need to reason about the whole pipeline rather than isolated models [23, 17]. This thesis therefore positions itself between component-level research and black-box managed services by focusing on the architecture needed to make a transparent, distributed speech translation system workable.

2.2 Microservices and ML Maintainability

Microservices are relevant to this thesis because they offer a way to separate concerns that scale differently and fail differently. Newman describes microservices as independently deployable services organized around business or domain capabilities, while Richardson emphasizes explicit boundaries, contracts, and asynchronous interaction patterns as mechanisms for preserving autonomy over time [22, 26]. For a speech translation platform, this matters because audio ingress, speech recognition, machine translation, speech synthe-

sis, orchestration, and persistence rarely share the same performance profile or operational lifecycle.

Maintainability becomes even more important when AI components are involved. Engineering AI systems requires attention not only to code modularity, but also to data movement, model packaging, runtime dependencies, reproducibility, and observability across model-serving boundaries [7]. Xu et al. further highlight that scalability and maintainability challenges in machine learning systems often arise from the interaction between model pipelines and their surrounding operational infrastructure, not from the model itself [31]. A microservice architecture does not automatically solve these problems, but it provides a structure in which concerns such as independent scaling, versioned interfaces, and controlled failure domains can be made explicit and studied.

The trade-off is that decomposition introduces coordination overhead. A microservice-based AI pipeline gains modularity and clearer ownership, but it also introduces network hops, serialization costs, and observability requirements that would be less visible in a monolith. This tension between modular maintainability and latency overhead is one of the central architectural trade-offs examined in the thesis.

2.3 Event-Driven Architecture (EDA)

Event-driven architecture is the coordinating paradigm adopted in this thesis because it decouples producers and consumers while allowing the pipeline to remain observable and extensible. In data-intensive systems, logs and streams provide a durable coordination mechanism that can support replay, asynchronous processing, and bounded service autonomy [13]. Enterprise integration patterns further describe how messages, envelopes, and broker-mediated routing can be used to connect heterogeneous components without forcing direct synchronous dependencies [10].

For speech translation, an event-driven design is attractive because the output of one stage naturally becomes the input of the next. Speech recognition emits text events, translation consumes and enriches those events, and synthesis produces output artifacts for return to the client. This pattern supports service isolation and flexible scaling, while also making it possible to trace processing across stage boundaries through correlation identifiers and hop-to-hop duration measurements. Brokered event streams and contract-driven event exchange are therefore not incidental implementation details; they are part of the architectural rationale for how a distributed AI pipeline can remain diagnosable under load [13, 26].

The same literature also motivates two specific patterns that matter later in the thesis. First, the *claim-check* pattern allows large artifacts such as audio files to be stored out of band while lightweight references move through the event stream [10]. In a secure system, those references must be bounded and

short-lived rather than permanent bearer credentials. Second, schema governance is needed so that event producers and consumers can evolve without breaking the pipeline. In the context of this thesis, schema evolution is treated as a high-level contract-governance concern: event changes must preserve interoperability and evidence traceability, not merely serialization success.

2.4 AI Pipeline Components

The platform examined in this thesis composes several AI-facing components into one end-to-end workflow. Automatic speech recognition converts audio into text, machine translation transfers meaning across languages, and text-to-speech converts translated text into audible output. Voice activity detection and ingress/orchestration logic support these stages by segmenting input, reducing unnecessary compute, and coordinating the conditions under which downstream services should act.

Each component family has its own research and engineering concerns. Whisper-style ASR emphasizes robustness across noisy and multilingual inputs [25]. Transformer-based MT and related sequence models provide the translation backbone used in many modern systems [5, 29]. Speech-to-text and direct speech-translation work show how model architectures can be tuned for latency and multilingual operation, but they do not remove the need for surrounding system decisions about batching, warm starts, queue discipline, and failure containment [30, 11, 32]. The architecture must therefore be studied as the environment in which these AI components become a usable platform.

This component view also supports the thesis’ maintainability argument. Treating ASR, MT, TTS, VAD, and orchestration as bounded but cooperating services makes it easier to reason about resource isolation, deployment strategy, and selective optimization. It also creates explicit integration points where observability and evidence collection can be attached without requiring the system to be redesigned as a single model-serving endpoint.

2.5 Challenges in Distributed Architectures

Distributed speech translation is constrained by more than raw model accuracy. Tail latency accumulates across network calls, queueing delays, serialization boundaries, and service startup behavior. A system that appears fast in isolated component benchmarks can still fail to feel real-time when exposed to concurrency, warm-up effects, or failure recovery paths. This is why the thesis treats P99 latency, rather than only average response time, as an architectural driver.

Scalability introduces a second challenge. Different pipeline stages scale according to different resource bottlenecks: ASR and TTS may be GPU-bound, orchestration may be coordination-bound, and persistence or broker infrastruc-

ture may become throughput-sensitive under burst load. Distributed-systems literature shows that scaling decisions are inseparable from partitioning strategy, observability, and failure semantics [13, 6, 15]. For this reason, the thesis evaluates scalability as a system property of the whole pipeline rather than as a proxy for the throughput of any single component.

Robustness and diagnosability form the third challenge. A multi-stage AI pipeline must continue to behave predictably when one stage slows down, restarts, or emits malformed data. Correlation identifiers, bounded message contracts, and hop-to-hop timing are essential because they allow failures to be localized and explained. At the same time, observability must remain privacy-safe. In this thesis, performance analytics are treated as metadata-only evidence: aggregated percentiles, event types, and duration summaries are admissible in the manuscript, while raw user content and raw request traces are not. That constraint is part of the research design rather than an afterthought.

2.6 Quality Assurance: From Testing to Evaluation

The background for this thesis must distinguish between testing a system and evaluating a research claim. Testing establishes whether the platform behaves as intended under controlled scenarios. Evaluation interprets the resulting evidence against the research questions and success criteria. In a distributed AI pipeline, both are necessary: without testing there is no trustworthy basis for evidence, and without evaluation there is no defensible thesis argument.

Reproducibility is therefore central. The project adopts a uv-first validation workflow so that later claims about latency, robustness, and integration behavior can be tied to repeatable execution rather than undocumented local setups. That workflow matters academically because it narrows the gap between implementation detail and thesis evidence. At the same time, only privacy-safe evidence belongs in the manuscript. This is why the evidence posture distinguishes between shareable metadata and non-shareable debug information, and why later empirical claims must be supported with aggregated measurements and sanitized artifacts rather than raw logs or secret-bearing command examples.

This distinction also clarifies the role of the later evaluation chapter. The background chapter can justify why reproducibility, observability, and privacy-safe evidence handling are necessary. It does not yet claim that the platform satisfies the target metrics; that remains an empirical question for later chapters.

2.7 Synthesis and Design Implications

The literature reviewed in this chapter supports a clear design direction. Speech translation is no longer a speculative application area. The research gap has

shifted from individual model capability toward architectural composition and empirical system behavior. Microservices provide maintainability and bounded ownership. Event-driven coordination supports loose coupling and diagnosability. Distributed-systems theory explains why latency, scalability, and robustness must be treated together rather than independently.

These observations motivate the design choices developed in the next chapter. The thesis therefore carries forward six implications: (1) the pipeline should be decomposed into explicit service stages; (2) event flow should be traceable across stage boundaries; (3) large artifacts should move through claim-check style references rather than bulk messages; (4) contract evolution must preserve interoperability and evidence continuity; (5) evaluation evidence must be reproducible; and (6) manuscript-facing evidence must remain privacy-safe and aggregated. Together, these implications connect the literature review to the architectural response and make the later chapter sequence answerable against the research questions introduced in Chapter 1.

3 Analysis and Design

3.1 Architectural Drivers

3.2 Quality Attribute Analysis

3.3 High-Level System Architecture

3.4 Data Modeling and Governance

3.5 Service Decomposition

3.6 Infrastructure and Deployment Strategy

4 Implementation

4.1 Orchestrator and Event Flow

4.2 Data Management and the Claim Check Pattern

4.3 Model Wrapping and "Warm-Loading"

4.4 Architectural Guardrails and VAD Integration

5 Evaluation

5.1 Real-Time Responsiveness (Latency)

5.2 Heterogeneous Scalability Testing

5.3 Robustness and Chaos Engineering

5.4 Architectural Fidelity (Pipeline Quality)

5.5 Modifiability Proof

6 Conclusion

6.1 Future Work

A Requirement Specifications

This appendix contains the complete list of Functional (FR) and Non-Functional (NFR) requirements defined for the Universal Speech Translation Platform.

A.1 Functional Requirements

ID	Description	Priority
FR-001	Session Management: The system must support creating, starting, and stopping a session.	MUST
FR-002	Language Support: The system must support dynamic language selection per session. MVP minimum: da, en, de, fr, es to English.	MUST
FR-003	Language Switching: The system should allow changing language pairs during an active session.	SHOULD
FR-004	Microphone Stream: The system must support live audio ingestion from a microphone via a gateway.	MUST
FR-005	Audio Upload: The system must support uploading audio chunks as an alternative to streaming.	MUST
FR-006	Segment & Streaming: The system must process both discrete segments and continuous streams.	MUST
FR-007	Partial ASR Results: ASR must produce intermediate (partial) transcriptions before the final result.	MUST
FR-008	Streaming Translation: The translation component must update translations continuously as text evolves.	MUST
FR-009	Streaming TTS: The system must generate and stream synthesized speech for translated text.	MUST
FR-010	Artifact Storage: The system must store artifacts (audio, transcripts) in object storage (MinIO) for auditability.	MUST
FR-011	Retention Policy: The system must support automatic deletion of artifacts after a configurable retention period.	MUST
FR-012	Event-Driven Orchestration: Inter-service communication must be asynchronous via events. The workflow must be coordinated by a Pipeline Orchestrator to ensure ordering and state tracking.	MUST
FR-013	Schema Governance: Events must be governed by schemas (e.g., Avro) with strict compatibility checks.	MUST

ID	Description	Priority
FR-014	Correlation: A correlation ID must be propagated through all stages to trace user requests.	MUST
FR-015	Retry & Degradation: Services must retry on temporary failures and degrade gracefully on permanent ones.	MUST
FR-016	Durability: The system must resume processing after a restart without losing queued events.	MUST
FR-017	Deduplication: The client/UI should handle potential duplicate events (at-least-once delivery).	SHOULD
FR-018	Docker Compose: The system must be deployable locally via Docker Compose for development.	MUST
FR-019	Docker Swarm: The system must be deployable to a Docker Swarm cluster.	MUST
FR-020	Horizontal Scaling: Each microservice must be capable of independent horizontal scaling.	MUST
FR-021	Ray Compute: Compute-heavy stages (ASR, TTS) should utilize Ray Actors for warm-loaded execution.	SHOULD
FR-022	Infrastructure as Code: Infrastructure should be provisioned via Pulumi (Post-MVP).	SHOULD
FR-023	Architectural Guardrails: The system should implement runtime guardrails (e.g., filtering empty transcriptions, hallucination mitigation) to prevent invalid AI outputs from reaching the user.	SHOULD
FR-024	VAD as Gatekeeper: The system must utilize VAD to segment human speech from silence before sending data to compute-heavy ASR components.	MUST

A.2 Non-Functional Requirements

ID	Description	Priority
NFR-001	Time-to-First-Output: The first partial result must be delivered within 3 seconds.	MUST
NFR-002	End-to-End Latency: Median processing time per segment must be < 5 seconds.	MUST
NFR-003	Scalability Test: The system should support up to 20 concurrent sessions in testing.	SHOULD
NFR-004	Observability: The system must provide metrics, tracing, and structured logging.	MUST
NFR-005	Privacy: Logs must be anonymized and free of PII (raw audio/transcripts).	MUST

ID	Description	Priority
NFR-006	Encryption: Traffic should be encrypted (TLS) in transit (Post-MVP Requirement).	SHOULD
NFR-007	Artifact Security: Internal event references to artifacts must not contain bearer secrets.	MUST
NFR-008	Resource Efficiency: Services should run on standard server hardware with GPU support.	SHOULD
NFR-009	Reproducibility: Performance measurements must be reproducible via documented scripts.	MUST
NFR-010	Quality Evaluation: Translation quality is evaluated via both automated metrics and human rating.	SHOULD

B Sequence Diagrams

This appendix provides detailed sequence diagrams illustrating the asynchronous interaction between microservices.

C Data Contracts (Avro Schemas)

This appendix documents the strict data contracts enforced across the microservice pipeline using Apache Avro. These schemas serve as the "Shared Contract Artifact" described in Section 3.4, ensuring type safety and backward compatibility.

All events inherit a common envelope structure defined in `BaseEvent.avsc` to guarantee consistent tracing via `correlation_id`.

C.1 Base Envelope

The base structure that all specific events must adhere to. This ensures that infrastructure-level concerns (tracing, timing, provenance) are decoupled from business payloads.

```
1 {
2   "type": "record",
3   "name": "BaseEvent",
4   "namespace": "ai.speech.shared.events",
5   "doc": "Common envelope for all MVP events.",
6   "fields": [
7     {"name": "event_id", "type": "string", "doc": "UUID for this
8       event"},
9     {"name": "event_type", "type": "string", "doc": "Logical event
10       type name"},
11     {"name": "correlation_id", "type": "string", "doc": "UUID shared
12       across pipeline"},
13     {"name": "timestamp", "type": {"type": "long", "logicalType": "
14       timestamp-millis"}, "doc": "Unix epoch ms"},
15     {"name": "source_service", "type": "string", "doc": "Originating
16       service name"}
17   ]
18 }
```

Listing C.1: BaseEvent.avsc

C.2 Audio Ingestion

The `AudioInputEvent` is emitted by the Ingress Gateway. Note the `audio_bytes` field for small payloads and `audio_uri` for the "Claim Check" pattern if payloads exceed Kafka limits.

```
1 {
2   "type": "record",
```



```

3   "name": "AudioInputEvent",
4   "namespace": "ai.speech.shared.events",
5   "fields": [
6     {"name": "event_id", "type": "string"},
7     {"name": "event_type", "type": "string"},
8     {"name": "correlation_id", "type": "string"},
9     {"name": "timestamp", "type": {"type": "long", "logicalType": "
      timestamp-millis"}},
10    {"name": "source_service", "type": "string"},
11    {
12      "name": "payload",
13      "type": {
14        "type": "record",
15        "name": "AudioInputPayload",
16        "fields": [
17          {"name": "audio_bytes", "type": ["null", "bytes"], "
            default": null},
18          {"name": "audio_uri", "type": ["null", "string"], "default
            ": null},
19          {"name": "audio_format", "type": "string", "doc": "e.g.,
            wav"},
20          {"name": "sample_rate_hz", "type": "int"},
21          {"name": "language", "type": ["null", "string"], "default"
            ": null, "aliases": ["language_hint"]},
22          {"name": "speaker_id", "type": ["null", "string"], "
            default": null}
23        ]
24      }
25    ]
26  ]
27 }

```

Listing C.2: AudioInputEvent.avsc

C.3 Speech Segmentation (VAD)

The `SpeechSegmentEvent` represents a cleaned, verified chunk of speech ready for ASR. It includes timing information (`start_ms`, `end_ms`) relative to the session start.

```

1 {
2   "type": "record",
3   "name": "SpeechSegmentEvent",
4   "namespace": "ai.speech.shared.events",
5   "fields": [
6     {"name": "event_id", "type": "string"},
7     {"name": "correlation_id", "type": "string"},
8     {
9       "name": "payload",
10      "type": {
11        "type": "record",
12        "name": "SpeechSegmentPayload",

```

```

13     "fields": [
14         {"name": "segment_id", "type": "string"},
15         {"name": "segment_index", "type": "int"},
16         {"name": "start_ms", "type": "long"},
17         {"name": "end_ms", "type": "long"},
18         {"name": "audio_bytes", "type": ["null", "bytes"], "
            default": null},
19         {"name": "segment_uri", "type": ["null", "string"], "
            default": null},
20         {"name": "sample_rate_hz", "type": "int"}
21     ]
22 }
23 }
24 ]
25 }

```

Listing C.3: SpeechSegmentEvent.avsc

C.4 Translation Output

The `TextTranslatedEvent` carries the core business value: the translated text. It includes source and target language codes to ensure downstream TTS picks the correct voice.

```

1 {
2   "type": "record",
3   "name": "TextTranslatedEvent",
4   "namespace": "ai.speech.shared.events",
5   "fields": [
6     {"name": "event_id", "type": "string"},
7     {"name": "correlation_id", "type": "string"},
8     {
9       "name": "payload",
10      "type": {
11        "type": "record",
12        "name": "TextTranslatedPayload",
13        "fields": [
14          {"name": "text", "type": ["null", "string"], "default":
              null},
15          {"name": "source_language", "type": "string"},
16          {"name": "target_language", "type": "string"},
17          {"name": "quality_score", "type": ["null", "float"], "
              default": null}
18        ]
19      }
20    }
21  ]
22 }

```

Listing C.4: TextTranslatedEvent.avsc

C.5 Synthesis Output (TTS)

The `AudioSynthesisEvent` is the final output. Similar to the input event, it supports both inline bytes and URIs via the Claim Check pattern to handle large generated audio files.

```
1 {
2   "type": "record",
3   "name": "AudioSynthesisEvent",
4   "namespace": "ai.speech.shared.events",
5   "fields": [
6     {"name": "event_id", "type": "string"},
7     {"name": "correlation_id", "type": "string"},
8     {
9       "name": "payload",
10      "type": {
11        "type": "record",
12        "name": "AudioSynthesisPayload",
13        "fields": [
14          {"name": "audio_bytes", "type": ["null", "bytes"], "default": null},
15          {"name": "audio_uri", "type": ["null", "string"], "default": null},
16          {"name": "duration_ms", "type": "long"},
17          {"name": "sample_rate_hz", "type": "int"},
18          {"name": "audio_format", "type": "string", "default": "wav"}
19        ]
20      }
21    ]
22  }
```

Listing C.5: `AudioSynthesisEvent.avsc`

Bibliography

- [1] Amazon Web Services. Amazon polly. <https://aws.amazon.com/polly/>, 2024. Accessed: 2026-01-26.
- [2] Amazon Web Services. Amazon transcribe. <https://aws.amazon.com/transcribe/>, 2024. Accessed: 2026-01-26.
- [3] Amazon Web Services. Amazon translate. <https://aws.amazon.com/translate/>, 2024. Accessed: 2026-01-26.
- [4] Apache Kafka. Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2023. Accessed: 2025-07-30.
- [5] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- [6] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 4 edition, 2021.
- [7] Len Bass, Qinghua Lu, Ingo Weber, and Liming Zhu. *Engineering AI Systems: Architecture and DevOps Essentials*. Addison-Wesley Professional, 2023.
- [8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, second edition, 2001.
- [9] Google Cloud. Cloud translation. <https://cloud.google.com/translate>, 2024. Accessed: 2026-01-26.
- [10] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2004.
- [11] Hirofumi Inaguma, Sravya Popuri, Gabriel Ilharco, Wei-Ning Hsu, Tal Remez, and Ann Lee. Unity: Two-pass direct speech-to-speech translation with discrete units. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15315–15339, 2023.
- [12] Yubin Kim, Ken Gu, Chanwoo Park, et al. Towards a science of scaling agent systems. *arXiv preprint arXiv:2512.08296*, 2025.

- [13] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [14] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.
- [15] Kubernetes. Kubernetes documentation. <https://kubernetes.io/docs/>, 2023. Accessed: 2025-07-30.
- [16] KUDO. 10 stats & breakthroughs in ai speech translation in 2024. <https://kudo.ai/blog/the-10-most-important-statistics-breakthroughs-in-ai-speech-translation-2024>. Accessed: 2025-07-30, Blog post.
- [17] B. P. Pradeep Kumar and Shreehari Menon. Real-time speech-to-speech translator: Analysis and implementation. In *2025 4th International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*. IEEE, 2025.
- [18] Tim Lindholm and Frank Yellin. *The JavaTM Virtual Machine Specification*. Addison-Wesley, 2nd edition, 1999.
- [19] Microsoft. Azure ai speech service. <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/>, 2024. Accessed: 2026-02-13.
- [20] Microsoft. Microsoft translator. <https://www.microsoft.com/translator/>, 2024. Accessed: 2026-01-26.
- [21] Mordor Intelligence. Speech to speech translation market size & share analysis. <https://www.mordorintelligence.com/industry-reports/speech-to-speech-translation>, 2024. Accessed: 2025-07-30.
- [22] Sam Newman. *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.
- [23] Shaphaba Oinam, Bharat Sharma, Penumalli Dindi Venkat, and Pushpendra Ratnam Verma. Ai driven voice translator: Enhancing multilingual communication through real-time translation. In *2025 12th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)*. IEEE, 2025.
- [24] Chen Qian, Zihao Xie, Yifei Wang, et al. Scaling large language model-based multi-agent collaboration. *arXiv preprint arXiv:2511.09525*, 2025.

- [25] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [26] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [27] Mohammad Sarim, Saim Shakeel, Laeeba Javed, et al. Direct speech to speech translation: A review. *arXiv preprint arXiv:2406.07155*, 2024.
- [28] UNESCO. Inclusion and education: All means all. <https://www.unesco.org/>, 2022. Accessed: 2026-01-26.
- [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [30] Changhan Wang, Anne Wu, Juan Pino, Alexei Baevski, Michael Auli, and Alexis Conneau. Fairseq s2t: Fast speech-to-text modeling with fairseq. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [31] Ping Xu, Aditi Sharma, and Jonas Müller. Scalability and maintainability challenges and solutions in machine learning: Systematic literature review. *arXiv preprint arXiv:2504.11079*, 2025.
- [32] Y. Zhang et al. Direct speech-to-speech translation: Architecture and optimization, 2024. Preprint.